AI Engineering and Claude Architect · Part IV, Java

AI in Java, Lesson 2

Streaming to Users: SSE and Virtual Threads

Users waited eleven seconds looking at a spinner, so many refreshed, which started the same expensive request again. The answer had been generating the whole time. Nobody could see it, and each refresh paid for it twice.

Module	AI08 of 13
Lesson	2 of 8
Prerequisites	AI08 L1, HTTP basics, and Java concurrency
Exam relevance	Out of scope
Reading time	~30 minutes

Contents

1. Streaming to Users

Lesson 2 of 8 · AI in Java

Streaming to Users

Key idea

Streaming doesn't make the answer faster. It makes the wait visible, and lets you stop when nobody is reading. Each stream holds a thread open for a long time while doing almost nothing, which is the case virtual threads were made for. **Remember that once streaming starts, the status is already 200, so errors have to travel as events.**

> *Before: the adapter (AI08 L1). Teaches: SSE with `SseEmitter`, the SDK's streaming API, virtual threads, cancelling on disconnect, and errors mid-stream. Used by: AI08 L8.*

The hook: three things streaming changes

Real output, from real Tomcat and real HTTP, with a scripted model:

```
content-type: text/event-stream
sse lines: [event:delta, data:Your , event:delta, data:order , event:delta, data:7781 ,
  event:delta, data:was , event:delta, data:delivered., event:done, data:end_turn]
stream failure: status=200 body=event:error data:retry
asked for 200 words, client left early: generated=3, a second later=3
```

Line	What it teaches
Events	The text arrives as it's generated
`status=200` with `event:error`	The status has gone. A later failure must be an event
`generated=3` of 200	When the client left, generation stopped

The last line needs care. It shows my server stopped pulling from the scripted model. A real stream must also close the HTTP connection to the API, which the SDK's `StreamResponse` does when it's closed. I haven't seen how much a real cancelled request bills, so check that.

Intuition: a live commentary

A radio commentator speaking as the match unfolds isn't making the match shorter. But listeners stay, and if they switch off, there's no reason to keep broadcasting to them. And if the stadium's power fails, the commentator has to say so on the air, since nobody can un-broadcast what's already gone out.

Mechanism: the path of a token

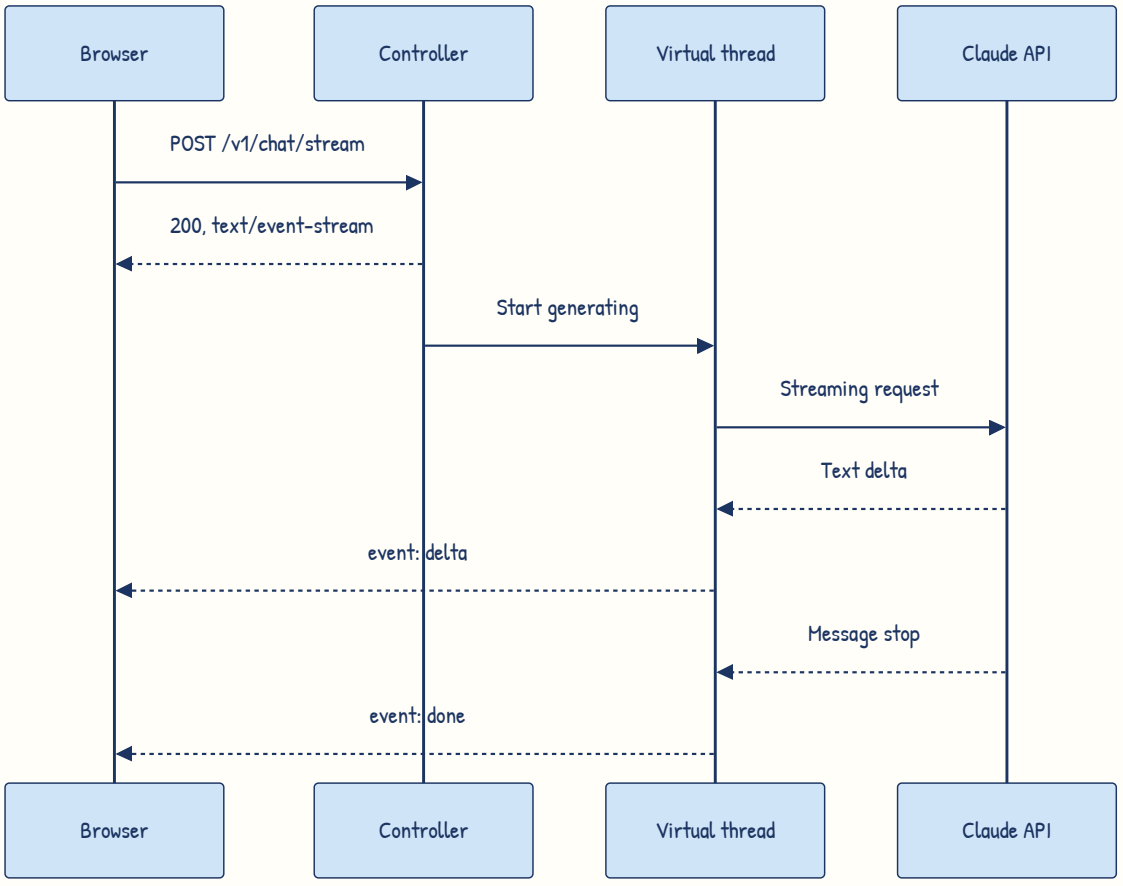


Figure 1. Streaming from the API to the browser. A virtual thread holds the request open, and each text fragment is forwarded as one SSE event.

SDK side

From the bundled Java reference, as used in `ClaudeModel.java`. It's tested against a fake that replays API-shaped SSE:

```
MessageAccumulator accumulator = MessageAccumulator.create();
try (StreamResponse<RawMessageStreamEvent> events =
        client.messages().createStreaming(params(request))) {
    events.stream().peek(accumulator::accumulate)
            .flatMap(event -> event.contentBlockDelta().stream())
            .flatMap(delta -> delta.delta().text().stream())
            .forEach(text -> onText.accept(text.text()));
}
return toTurn(accumulator.message());
```

Line	Why it matters
`try` with resources	Closing it ends the connection
`MessageAccumulator`	You still get the complete message, for usage and tool calls
Filtering to text deltas	Tool input and thinking arrive as other event types

Real output:

```
stream fragments: [Your , order , arrived.]
stream turn: stop=end_turn text=Your order arrived. out=12
```

Check `stop_reason` at the end, even when streaming. A stream ending in `max_tokens` looks like a sentence that just stopped, so tell the user.

Server side

From `ChatController.java`. The controller returns quickly, and the virtual thread does the waiting:

```
SseEmitter emitter = new SseEmitter(STREAM_TIMEOUT_MILLIS);
AtomicReference<Future<?>> work = new AtomicReference<>();
// If the user goes away, stop generating: nobody is reading, and tokens cost money.
emitter.onTimeout(() -> cancel(work));
emitter.onError(error -> cancel(work));
work.set(streams.submit(() -> generate(request, emitter)));
return emitter;
```

And the failure case:

```
} catch (ModelException e) {
    // Headers have gone already, so the error has to travel as an event.
    send(emitter, "error", e.kind().retryable ? "retry" : "failed");
    emitter.complete();
```

Event	Client should
`delta`	Append the text
`done`	Stop, and check the reason
`error`	Show a message, and keep the partial text clearly marked
No `done`, and the connection closed	**Treat it as failed**

The last row matters: a client that treats any end of stream as success will show cut-off answers as complete.

Why virtual threads fit

A stream spends nearly all its time waiting. Here's a lab holding many waits open. Each task only sleeps for 2 seconds, so **this measures how threads wait, on this laptop, and isn't a server benchmark**. Real output:

```
platform pool of 200       tasks=2000   peak concurrent=200    total=20.4s
virtual threads            tasks=2000   peak concurrent=2000   total=2.1s
virtual threads            tasks=20000  peak concurrent=20000  total=2.2s
```

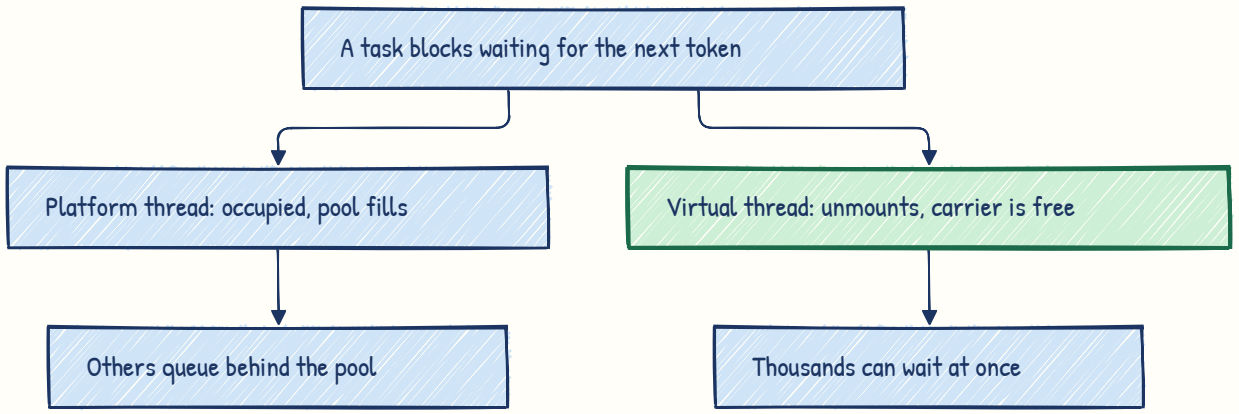


Figure 2. What a blocking wait costs. A platform thread stays occupied while it waits, and a virtual thread gives its carrier back.

Enable it for Spring with this, per the Boot docs:

```
spring:
  threads:
    virtual:
      enabled: true
```

Virtual threads remove the thread limit, and other limits take over:

New limit	What to do
API rate limits	A semaphore or gateway. 20,000 streams would hit limits long before threads
Database connections	Virtual threads don't add connections, so a pool still queues
Memory	Each stream buffers text
Sockets and proxies	Check file descriptor and proxy limits
CPU-heavy work	No benefit

On pinning: in JDK 21, blocking while inside a `synchronized` block can pin a virtual thread to its carrier, and later JDKs relax this. I state this from general knowledge, since the Boot page doesn't cover it. I haven't tested OkHttp or the SDK for pinning under load, so use JFR's `jdk.VirtualThreadPinned` event if you depend on this.

Why the obvious fixes fall short

Tempting move	Why it disappoints	Better
A spinner and a long timeout	Users refresh, and you pay twice	Stream
WebFlux everywhere for streaming	Rewrites your stack in reactive style	`SseEmitter` and virtual threads suit a servlet app
WebSockets	Two-way, with more state and proxy trouble	SSE, unless you need client messages mid-stream
Larger thread pool	Memory per thread, and it's still a cap	Virtual threads
Assuming end of stream means success	Cut-off answers look complete	Require a `done` event
Ignoring disconnects	You pay for text nobody reads	Cancel

Spring AI's docs say its streaming needs the reactive stack, as AI07 L4 noted, which matters if you use that instead.

How the field approaches it

Point	Source
Use streaming for long input, long output or high `max_tokens`, to avoid request timeouts	Claude API reference
`spring.threads.virtual.enabled` configures Boot's default executor and scheduler	Spring Boot docs
Time to first token is what users feel. LinkedIn stressed streaming, and an async pipeline	LinkedIn write-up (AI05 L1)

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: wait, then return.

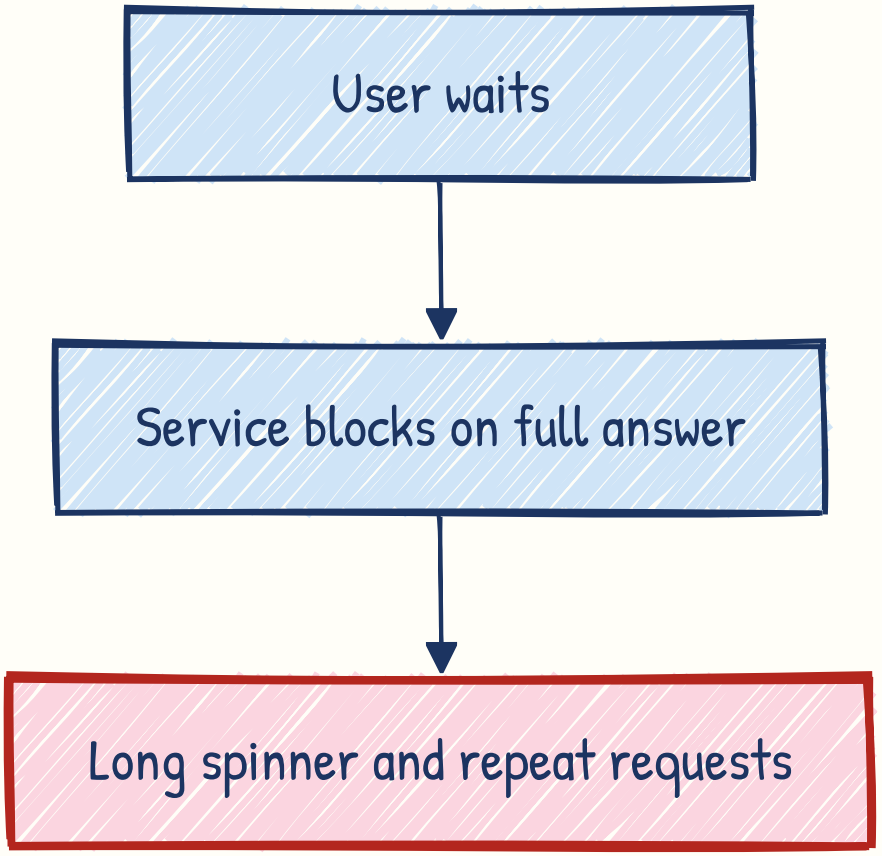


Figure 3. Baseline. Nothing reaches the user until the full answer is ready, so they refresh and pay again.

Improved: direct SSE. The wait is visible. New problem: a dropped connection loses the answer, and proxies may buffer the stream so that it isn't really streaming.

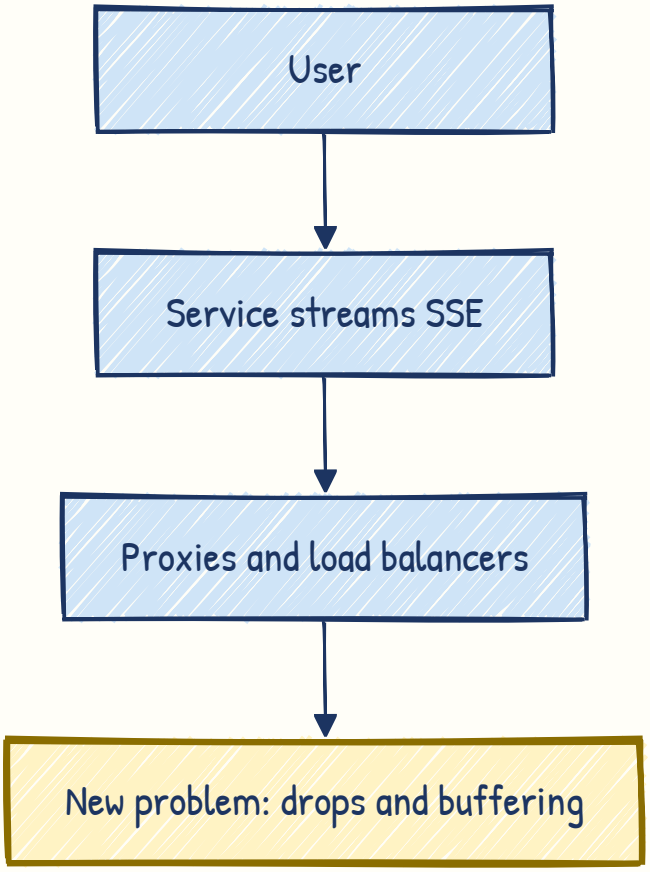


Figure 4. Improved. Text flows immediately, but connections drop, and proxies can hold everything back.

Production: resumable streams.

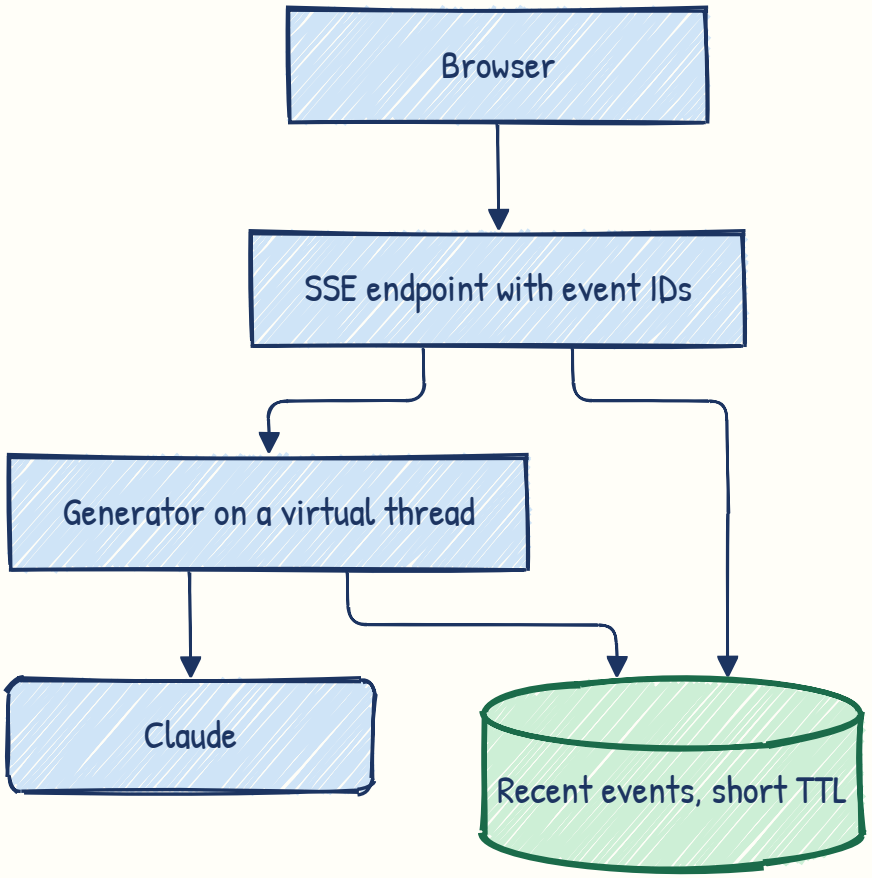


Figure 5. Production. Text is appended to a short-lived store with event IDs, so a reconnecting client resumes where it stopped. Heartbeats keep proxies from closing idle connections.

Component	Why it exists	How it communicates
Event IDs	Resume after a drop	`Last-Event-ID` header on reconnect
Event store	Holds recent output	Redis or similar, with TTL
Heartbeat	Idle proxies close quiet connections	SSE comment every 15 seconds or so
Concurrency limit	Protect quota	Semaphore per feature
Proxy settings	Must not buffer	For example, disable buffering on that route. Check your proxy

The event store, resume and heartbeat aren't built in `code/ai08-java`. They're a design.

Failures

Failure	Containment
Client disconnects	Cancel, so you don't keep paying
API fails mid-stream	`error` event, with partial text marked
`max_tokens` reached	Tell the user it was cut off
Proxy buffers	Test through the real path, not just locally
Emitter timeout	`onTimeout` cancels the work
Too many streams	Reject with 429 before starting

Trade-offs

Decision	Gain	Cost
SSE	Simple, one-way, plain HTTP	No client messages mid-stream
Virtual threads	Simple blocking code at scale	Pinning to watch, other limits to manage
Cancel on disconnect	Saves money	A brief disconnect loses the run, unless it's resumable
Resumable streams	Survives drops	A store and more logic

When not to stream: JSON for another service, or background work.

Common mistakes

1. **Treating end of stream as success.** Require a `done` event.
2. **Relying on status codes after streaming starts.** Send error events.
3. **Ignoring `stop_reason` in a stream.** `max_tokens` looks like a sudden stop.
4. **Not closing the stream.** Use try-with-resources.
5. **Thinking virtual threads remove all limits.** Rate limits and databases remain.
6. **Testing only on localhost.** Proxies behave differently.
7. **Holding a database connection during a stream.** Release it first.

What to say in an interview

"I'd stream with SSE, and hold each request on a virtual thread, since a stream mostly waits. Virtual threads remove the thread cap, so I'd add a semaphore for rate limits. Once streaming starts, the status is gone, so errors are events, and the client treats a missing done event as failure. On disconnect I cancel, because nobody is reading, and tokens cost money."

Certification lens

How this lesson maps to CCAR-F

Not tested.

These are original practice questions for understanding, not real or leaked exam questions.

Q Q1 · Debugging · Medium. Locally, tokens arrive one by one. In staging, nothing shows for 12 seconds, then it all arrives at once. The code is the same. What do you check?

A A proxy or load balancer buffering the response, or compression that holds output back. Disable buffering on that route, check that SSE headers survive, and test through the real path. It isn't the model, since the local run streams.

Q Q2 · Best decision · Medium. A team moved to virtual threads, and now under load they see Postgres connection timeouts and 429s from the API. What happened, and what would you change?

A The thread cap used to limit concurrency, and hid these limits. Thousands of tasks now reach the connection pool and the API at once. Add explicit limits with semaphores, keep the connection pool sized for the database, don't hold connections during model calls, and back off on 429.

Q Q3 · Code analysis · Hard. A client's code is `stream.onclose = () => showAnswer(text)`. What goes wrong when the API fails halfway? Fix it.

A A partial answer is shown as complete, and that could be a half-finished instruction. Show as complete only on a `done` event with a normal stop reason. Otherwise mark it as incomplete and offer a retry.

Revision - Lesson 2

→ Streaming makes the wait visible, and lets you stop early.

→ SDK: createStreaming, with a MessageAccumulator for the final message.

→ Filter to text deltas, and check stop_reason at the end.

→ After headers go, errors must be events.

→ Clients should require a done event.

→ Cancel on disconnect, since nobody is reading.

→ SseEmitter on a virtual thread fits servlet apps.

→ In the lab, 200 platform threads took 20.4s for 2,000 sleeping tasks, and virtual threads took 2.1s.

→ Virtual threads remove the thread cap, so add limits for rate and connections.

→ Watch for pinning on JDK 21.

→ Test through real proxies, and add heartbeats and resume in production.

Your notes